



IIC2343 - Arquitectura de Computadores (II/2026)

Ayudantía 1

Ayudantes: Ignacio Gajardo (gajardo.ignacio@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl),
Nicolás Romo Aubele (nroma@uc.cl)

Pregunta 1: Representación de Números

(a) Convierta los siguientes números decimales a binario:

1. 79_{10}
2. 401_{10}

Solución:

Para pasar los números a binario usaremos la siguiente tabla de potencias de 2.

2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125	0.0625

1. 78_{10} :

$$79 - 2^6 = 79 - 64 = 15$$

$$15 - 2^3 = 15 - 8 = 7$$

$$7 - 2^2 = 7 - 4 = 3$$

$$3 - 2^1 = 3 - 2 = 1$$

$$1 - 2^0 = 1 - 1 = 0$$

Entonces,

$$79 = 2^6 + 2^3 + 2^2 + 2^1 + 2^0$$

$$97_{10} = 1100001_2$$

2. 401_{10} :

$$401 - 2^8 = 401 - 256 = 145$$

$$145 - 2^7 = 145 - 128 = 17$$

$$17 - 2^4 = 17 - 16 = 1$$

$$1 - 2^0 = 1 - 1 = 0$$

Entonces,

$$401 = 2^8 + 2^7 + 2^4 + 2^0$$

$$401_{10} = 110010001_2$$

(b) Reescriba los siguientes números según su representación complemento de 2. (ocupe grupo de 4 bits).

1. -5_{10}

2. $B2_{16} = 0xB2$

3. -123_{10}

4. -1_{10}

5. 0_n

Solución:

1. -5_{10} :

Pasamos el número sin signo a binario,

$$5_{10} = 101_2$$

Completamos el grupo de 4 bits con 0's.

$$101_2 = 0101_2$$

Para complemento de 2 debemos invertir los bits y después sumar 1.

$$0101 \rightarrow 1010 \rightarrow 1011$$

$$-5_{10} = 1011_2 \quad (\text{complemento de 2})$$

2. $B2_{16}$:

Pasamos el número a binario,

Recomendamos separar cada caracter en 4 bits y luego transformarlos individualmente.

$$B2_{16} = 1011 \ 0010_2$$

Como el bit más significativo es 1 y el número es positivo, debemos agregar un cero a la izquierda para mantener el signo correcto. Sin embargo, como se nos exige hacerlo con grupo de 4 bits debemos agregar un grupo entero de 4 bits.

$$1011\ 0010_2 = 0000\ 1011\ 0010_2$$

Para representar el número en complemento de 2 no debemos hacer nada más porque es un número positivo.

$$B_{2_{16}} = 0000\ 1011\ 0010_2 \quad (\text{complemento de 2})$$

3. -116_{10} :

Pasamos el número sin signo a binario,

$$116_{10} = 111\ 0100_2$$

Completamos el grupo de 4 bits con 0's a la izquierda.

$$01110100_2 = 0111\ 0100_2$$

Para complemento de 2 debemos invertir los bits y después sumar 1.

$$01110100 \rightarrow 10001011 \rightarrow 10001100$$

$$-116_{10} = 10001100_2 \quad (\text{complemento de 2})$$

4. -1_{10} :

Pasamos el número sin signo a binario,

$$1_{10} = 1_2$$

Completamos el grupo de bits con 0's.

$$1_2 = 0000\ 0001_2$$

Para complemento de 2 debemos invertir los bits y después sumar 1.

$$00000001 \rightarrow 11111110 \rightarrow 11111111$$

$$-1_{10} = 11111111_2 \quad (\text{complemento de 2})$$

5. 0_n :

Pasamos el número sin signo a binario,

$$0_n = 0_2$$

Completamos (8 bits) con 0's.

$$0_2 = 00000000_2$$

En complemento de 2 solo hay una representación.

$$0_{10} = 00000000_2 \quad (\text{complemento de 2})$$

- (c) ¿Qué ocurre si sumamos 57_{10} con 17_{10} usando solo 7 bits y representándolos en complemento 2?

Solución:

$$57_{10} = 0111001_2$$

$$17_{10} = 0010001_2$$

$$0111001_2 + 0010001_2 = 1001010_2$$

$$1000111_2 = -54_{10}$$

El resultado real debería ser 74_{10} , pero no cabe en 7 bits con complemento de 2, por lo que ocurre **overflow**.

Pregunta 2: Complemento de 2

- (a) **(I1 2024-2)** Al utilizar complemento de 2 para representaciones posicionales binarias, se genera una representación no equilibrada donde existen más elementos negativos que positivos. Modifique el algoritmo de transformación tal que ahora existan más números positivos que negativos.

Solución: Una modificación posible consiste en añadir un paso previo al algoritmo de complemento de 2, como se muestra a continuación:

- “Anteponer” un 1 al número (*i.e.* cambiar su bit más significativo a 1).
- Negar todos los bits.
- Sumar 1.

Con este cambio, se tendrá una mayor cantidad de números positivos que de negativos, además de respetar el resultado de la suma entre un número y su inverso aditivo. Se otorga **1 pto.** por proponer una modificación del algoritmo, y **1 pto.** si la propuesta es correcta, *i.e.* genera más positivos que negativos.

- (b) **(I1 2023-2)** Un estudiante implementa un *script* para computar el inverso aditivo de un número en base binaria representado en complemento de 2. Para verificar su correctitud, imprime la suma entre un número y su inverso aditivo. No obstante, el resultado, interpretado como número entero en base binaria, es igual a -1. Indique qué error cometió el estudiante y cómo se puede resolver.

Solución: En este caso, si -1 se interpretó como número entero en base binaria, este corresponde a la secuencia binaria que solo posee bits iguales a uno. Esto ocurre en particular al realizar la suma de un número con su inverso computado con **complemento de 1** (la negación de todos los bits del número original). Para resolverlo, basta con incrementar en una unidad al inverso aditivo computado para que la suma resultante sea igual a cero (descartando el bit de *carry* generado) y el *script* imprima el cero esperado. Se otorga **1 pto.** por señalar correctamente el error y **1 pto.** por explicar cómo se resuelve.

Pregunta 3: Operaciones en binario

- (a) Dadas las variables $A = 0xDEF$ y $B = 0x54$, ambas utilizando representación de complemento a 2. Pase las variables a binario, identifique el bit de signo de cada una y calcule la operación $A + B$. El resultado debe expresarse en formato hexadecimal utilizando la cantidad de bits del operando más grande.

Solución: Primero, identificamos los signos y pasamos a binario:

- $A = 0xDEF \rightarrow 1101\ 1110\ 1111_2$. El bit más significativo (MSB) es **1**, por lo tanto, es un número **negativo**.
- $B = 0x54 \rightarrow 0101\ 0100_2$. El MSB es **0**, por lo tanto, es un número **positivo**.

Para realizar la suma $A + B$, ambas secuencias deben tener 12 bits. Como B es positivo, la extensión de signo se realiza rellenando con ceros a la izquierda:

- $B_{\text{extendido}} = 0000\ 0101\ 0100_2 = 0x054$.

Realizamos la suma binaria de forma horizontal:

$$1101\ 1110\ 1111_2 + 0000\ 0101\ 0100_2 = 1110\ 0100\ 0011_2$$

Pasamos el resultado a hexadecimal agrupando de a 4 bits: $1110 \rightarrow E$, $0100 \rightarrow 4$, $0011 \rightarrow 3$. El resultado final es **0xE43**.

- (b) Se tienen dos variables en 8 bits utilizando representación de Complemento a 2: $X = 0xE5$ y $Y = 0x1A$. Calcule $X - Y$ y entregue el resultado final en hexadecimal. Además, demuestre transformando los valores a base decimal que su resultado binario tiene sentido matemático.

Solución: Primero, transformamos a binario:

- $X = 0xE5 \rightarrow 1110\ 0101_2$ (MSB 1, negativo).
- $Y = 0x1A \rightarrow 0001\ 1010_2$ (MSB 0, positivo).

Para calcular $X - Y$, lo reescribimos como suma $X + C_2(Y)$:

- $C_2(Y) = C_2(0001\ 1010_2) = 1110\ 0110_2$.

Realizamos la suma binaria:

$$1110\ 0101_2 + 1110\ 0110_2 = \mathbf{1}\ 1100\ 1011_2$$

El noveno bit (el **1** del extremo izquierdo) es el *carry out* y **se descarta**. Nos quedamos con $1100\ 1011_2$, que en hexadecimal es **0xCB**.

Demostración y análisis:

- Valor decimal de X : $C_2(1110\ 0101_2) = 0001\ 1011_2 = 27_{10} \rightarrow X = -27_{10}$.
- Valor decimal de Y : Leído directamente al ser positivo $\rightarrow Y = 26_{10}$.
- Operación real esperada: $-27 - 26 = -53_{10}$.
- Verificación del resultado: $1100\ 1011_2$ es negativo. Su magnitud es $C_2(1100\ 1011_2) = 0011\ 0101_2 = 53_{10}$. Representa al **-53**, lo que comprueba el proceso.
- **Overflow: No ocurrió.** Se sumaron dos números negativos y el resultado fue negativo, respetando el rango de los 8 bits.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

